



Universidad Tres Culturas

Universidad Tres Culturas “toreo”

Trabajo: Proyecto Integrador Parcial 3

Asignatura: Estructura de Datos

Nombre del alumno (a): Hernández Venancio Isaac Jair

Matricula: 250146665

Grupo: 5SLEINSCMA

Profesor: Angel Uriel Morales Cardoso

Carrera: Ingeniería en Sistemas Computacionales

CUATRIMESTRE 2026

1. Introducción

El presente proyecto corresponde al tercer parcial de la asignatura Estructura de Datos. Consiste en actualizar el Sistema de Gestión de Cursos UTC desarrollado durante los parciales anteriores, incorporando nuevas estructuras de datos y algoritmos.

En esta versión se integran un árbol binario de búsqueda, un grafo para representar relaciones entre cursos, diferentes métodos de ordenamiento y métodos de búsqueda. El sistema continúa funcionando mediante consola y conserva como elemento principal la clase `Curso`.

El proyecto tiene como finalidad aplicar los conocimientos adquiridos durante el curso y comparar diferentes estructuras y algoritmos para organizar, relacionar y localizar información.

2. Objetivo

Actualizar el Sistema de Gestión de Cursos UTC mediante la integración de árboles binarios de búsqueda, grafos, métodos de ordenamiento y métodos de búsqueda en Java.

El propósito es utilizar diferentes estructuras y algoritmos para organizar, relacionar y localizar información dentro del sistema.

3. Descripción general

El Sistema de Gestión de Cursos UTC 3.0 es una aplicación desarrollada en Java que funciona completamente desde consola.

El sistema permite administrar cursos mediante diferentes operaciones, entre ellas:

- Agregar cursos.
- Mostrar cursos registrados.
- Eliminar cursos.
- Inscribir estudiantes.
- Dar de baja estudiantes.
- Insertar cursos en un árbol binario de búsqueda.
- Buscar cursos mediante el árbol.
- Mostrar el recorrido inorden.
- Crear relaciones entre cursos mediante un grafo.
- Mostrar la matriz de adyacencia.
- Ordenar cursos mediante diferentes algoritmos.
- Realizar búsquedas secuenciales y binarias.
- Consultar el historial de acciones.

Además, se incorporaron funcionalidades adicionales para comparar el número de pasos de las búsquedas y mostrar los cursos que cuentan con cupo disponible.

4. Clase principal Curso

La clase **Curso** funciona como la entidad central del sistema.

Cuenta con los siguientes atributos:

Atributo	Tipo	Descripción
idCurso	int	Identificador único del curso.
clave	String	Clave del curso.
nombre	String	Nombre del curso.
docente	String	Docente asignado.
cupoMaximo	int	Cantidad máxima de estudiantes.
numeroInscritos	int	Cantidad actual de estudiantes inscritos.

El atributo **idCurso** es utilizado como criterio principal para el árbol binario de búsqueda y para las búsquedas.

La clase también contiene métodos para:

- Inscribir estudiantes.
- Dar de baja estudiantes.
- Consultar el cupo disponible.
- Mostrar la información del curso.

5. Estructura del proyecto

El proyecto está organizado mediante diferentes clases, cada una con una responsabilidad específica.

Clase

Función

<code>Main.java</code>	Contiene el menú principal y controla las operaciones.
<code>Curso.java</code>	Representa los cursos registrados.
<code>NodoArbolCurso.java</code>	Representa los nodos del árbol binario.
<code>ArbolCursos.java</code>	Administra el árbol, inserción, búsqueda e inorden.
<code>GrafoCursos.java</code>	Administra las relaciones entre cursos.
<code>Ordenamientos.java</code>	Contiene los métodos de ordenamiento.
<code>Busquedas.java</code>	Contiene los métodos de búsqueda.
<code>HistorialAcciones.java</code>	Registra las acciones realizadas en el sistema.

6. Árbol binario de búsqueda

El sistema implementa un árbol binario de búsqueda para organizar y localizar cursos mediante el atributo `idCurso`.

Cada nodo del árbol almacena un objeto de tipo `Curso` y cuenta con dos referencias:

- Hijo izquierdo.
- Hijo derecho.

La regla utilizada para insertar los cursos es:

- Si el ID del nuevo curso es menor que el nodo actual, se coloca a la izquierda.
- Si el ID es mayor, se coloca a la derecha.

La clase `ArbolCursos` permite realizar las siguientes operaciones:

Inserción

Los cursos se insertan utilizando `idCurso` como criterio.

Búsqueda

El árbol compara el ID buscado con el nodo actual y decide si debe continuar por la izquierda o por la derecha.

Recorrido inorden

El recorrido inorden visita:

1. Subárbol izquierdo.
2. Nodo actual.
3. Subárbol derecho.

Por esta razón, los cursos aparecen ordenados de menor a mayor según su **idCurso**.

7. Grafo de cursos

El sistema utiliza un grafo para representar relaciones entre diferentes cursos.

En este grafo:

- Cada curso representa un vértice.
- Una relación entre dos cursos representa una arista.
- El grafo es dirigido.
- La representación utilizada es una matriz de adyacencia.

Una relación puede representar, por ejemplo, que un curso es prerequisite de otro.

La matriz utiliza:

1 = existe una relación.

0 = no existe una relación.

Por ejemplo:

	101	102	105
101	0	1	0
102	0	0	1
105	0	0	0

En este ejemplo, el curso 101 tiene una relación con el 102 y el curso 102 tiene una relación con el 105.

8. Métodos de ordenamiento

El proyecto implementa los métodos de ordenamiento obligatorios establecidos para el parcial.

8.1 Bubble Sort directo

Ordena los cursos de menor a mayor utilizando comparaciones e intercambios entre elementos consecutivos.

8.2 Bubble Sort inverso

Realiza el mismo procedimiento, pero ordenando los cursos de mayor a menor.

8.3 Inserción directa

Este método toma cada elemento y lo coloca en la posición correspondiente dentro de la parte que ya se encuentra ordenada.

8.4 Selección directa

Busca el elemento menor dentro de la parte restante de la lista y lo coloca en la posición correspondiente.

En el proyecto estos métodos utilizan el `idCurso` como criterio de ordenamiento.

9. Métodos de búsqueda

El sistema implementa dos métodos principales de búsqueda.

9.1 Búsqueda secuencial

La búsqueda secuencial recorre los cursos uno por uno hasta encontrar el ID solicitado.

Si encuentra el curso, devuelve la información correspondiente.

Si llega al final sin encontrarlo, muestra un mensaje indicando que no existe.

9.2 Búsqueda binaria

La búsqueda binaria divide el espacio de búsqueda utilizando el elemento central.

Para poder utilizar este método, los cursos deben encontrarse previamente ordenados.

En el sistema, los cursos se ordenan por **idCurso** antes de realizar la búsqueda binaria.

10. Menú principal

El sistema cuenta con el siguiente menú:

```
=====
      SISTEMA DE GESTIÓN DE CURSOS UTC 3.0
=====
1. Agregar curso
2. Mostrar cursos
3. Eliminar curso
4. Inscribir estudiante
5. Dar de baja estudiante
6. Insertar cursos en árbol binario
7. Buscar curso en árbol binario
8. Mostrar recorrido inorden del árbol
9. Crear relación entre cursos (grafo)
10. Mostrar grafo / matriz de adyacencia
11. Ordenar con Bubble Sort directo
12. Ordenar con Bubble Sort inverso
13. Ordenar con inserción directa
14. Ordenar con selección directa
15. Búsqueda secuencial
16. Búsqueda binaria
17. Mostrar historial de acciones
18. Comparar pasos: secuencial vs binaria
19. Mostrar cursos con cupo disponible
20. Salir
=====
```

11. Validaciones

El sistema contiene diferentes validaciones para evitar errores durante su funcionamiento.

No se permite registrar un curso cuando su **idCurso** ya existe.

No se permite registrar una clave que ya se encuentre registrada.

No se permite inscribir estudiantes cuando el curso alcanzó su cupo máximo.

No se permite dar de baja estudiantes cuando el curso tiene cero inscritos.

Cuando una búsqueda no encuentra resultados, el sistema muestra un mensaje informativo.

Si no existen cursos registrados, el sistema informa al usuario.

Antes de utilizar la búsqueda binaria, los cursos son ordenados utilizando el mismo criterio de búsqueda.

12. Historial de acciones

El sistema cuenta con una clase llamada `HistorialAcciones`.

Esta clase registra las operaciones realizadas por el usuario.

Algunas acciones que pueden registrarse son:

- Agregar un curso.
- Eliminar un curso.
- Inscribir un estudiante.
- Dar de baja un estudiante.
- Realizar una búsqueda.
- Ejecutar un método de ordenamiento.
- Crear una relación en el grafo.
- Comparar métodos de búsqueda.

El usuario puede consultar el historial mediante la opción correspondiente del menú.

13. Funcionalidades adicionales

Como funcionalidad adicional se implementó la comparación del número de pasos entre la búsqueda secuencial y la búsqueda binaria.

El usuario introduce un ID y el sistema muestra cuántos pasos necesitó cada método para realizar la búsqueda.

Ejemplo:

===== COMPARACIÓN DE BÚSQUEDAS =====

Secuencial: 5 pasos.

Binaria: 2 pasos.

Resultado: curso encontrado.

También se agregó una función para mostrar los cursos que todavía cuentan con cupo disponible.

Esta función muestra la información del curso y el número de lugares restantes.

14. Casos de prueba

De acuerdo con los requisitos del proyecto, se deben realizar diferentes pruebas.

Caso 1: Registrar cursos

Registrar al menos cinco cursos y comprobar que aparecen correctamente en el sistema.

Caso 2: Buscar cursos

Realizar una búsqueda de un curso existente y posteriormente buscar un ID inexistente.

Caso 3: Árbol binario

Insertar los cursos en el árbol y mostrar el recorrido inorden.

Caso 4: Búsqueda en árbol

Buscar un curso utilizando su `idCurso`.

Caso 5: Grafo

Crear al menos cuatro relaciones entre diferentes cursos.

Caso 6: Matriz de adyacencia

Mostrar la matriz generada por las relaciones creadas.

Caso 7: Bubble Sort

Ejecutar Bubble Sort directo y Bubble Sort inverso.

Caso 8: Otros ordenamientos

Ejecutar inserción directa y selección directa.

Caso 9: Búsquedas

Ejecutar búsqueda secuencial y búsqueda binaria.

Caso 10: Historial

Mostrar las acciones realizadas durante las pruebas.

15. Evidencias de funcionamiento

Para complementar el documento se deben agregar capturas de pantalla del funcionamiento del programa.

Captura 1. Menú principal

```
=====
SISTEMA DE GESTIÓN DE CURSOS UTC 3.0
=====
1.  Agregar curso
2.  Mostrar cursos
3.  Eliminar curso
4.  Inscribir estudiante
5.  Dar de baja estudiante
6.  Insertar cursos en árbol binario
7.  Buscar curso en árbol binario
8.  Mostrar recorrido inorden del árbol
9.  Crear relación entre cursos (grafo)
10. Mostrar grafo / matriz de adyacencia
11. Ordenar con Bubble Sort directo
12. Ordenar con Bubble Sort inverso
13. Ordenar con inserción directa
14. Ordenar con selección directa
15. Búsqueda secuencial
16. Búsqueda binaria
17. Mostrar historial de acciones
18. Comparar pasos: secuencial vs binaria
19. Mostrar cursos con cupo disponible
20. Salir
```

Captura 2. Cursos registrados

```
=====
Seleccione una opción: 1
ID del curso: 0001
Clave: 0100
Nombre: Batallas de aura
Docente: Chris
Cupo máximo: 2
Curso agregado correctamente.
```

Captura 3. Árbol binario

```
=====
Seleccione una opción: 8
ID: 1 | Clave: 0100 | Nombre: Batallas de aura | Docente: Chris | Cupo: 2 | Inscritos: 1 | Disponibles: 1
ID: 2 | Clave: 0200 | Nombre: Creeper vs zombie | Docente: Las epicas batallas del rap del friquismo | Cupo: 2 | Inscritos: 2 | Disponibles: 0
ID: 3 | Clave: 0300 | Nombre: Homelo chino | Docente: Homelo chino | Cupo: 1 | Inscritos: 1 | Disponibles: 0
ID: 4 | Clave: 0450 | Nombre: Carniceria chapala | Docente: Chapala | Cupo: 15 | Inscritos: 1 | Disponibles: 14
ID: 25 | Clave: 0350 | Nombre: La hora pico | Docente: Eugenio Terbez | Cupo: 15 | Inscritos: 0 | Disponibles: 15
ID: 101 | Clave: P00101 | Nombre: Programación Orientada a Objetos | Docente: Ing. Miguel | Cupo: 25 | Inscritos: 0 | Disponibles: 25
ID: 102 | Clave: P0102 | Nombre: Programación II | Docente: Ing. Carlos | Cupo: 25 | Inscritos: 1 | Disponibles: 24
ID: 105 | Clave: B0105 | Nombre: Bases de Datos | Docente: Mtra. Laura | Cupo: 30 | Inscritos: 2 | Disponibles: 28
ID: 108 | Clave: E0108 | Nombre: Estructura de Datos | Docente: Ing. Ana | Cupo: 30 | Inscritos: 2 | Disponibles: 28
ID: 110 | Clave: V00110 | Nombre: Desarrollo Web | Docente: Mtra. Sofia | Cupo: 20 | Inscritos: 0 | Disponibles: 20
```

Captura 4. Grafo

```
=====
Seleccione una opción: 8
ID: 1 | Clave: 0100 | Nombre: Batallas de aura | Docente: Chris | Cupo: 2 | Inscritos: 1 | Disponibles: 1
ID: 2 | Clave: 0200 | Nombre: Creeper vs zombie | Docente: Las epicas batallas del rap del friquismo | Cupo: 2 | Inscritos: 2 | Disponibles: 0
ID: 3 | Clave: 0300 | Nombre: Homelo chino | Docente: Homelo chino | Cupo: 1 | Inscritos: 1 | Disponibles: 0
ID: 4 | Clave: 0450 | Nombre: Carniceria chapala | Docente: Chapala | Cupo: 15 | Inscritos: 1 | Disponibles: 14
ID: 25 | Clave: 0350 | Nombre: La hora pico | Docente: Eugenio Terbez | Cupo: 15 | Inscritos: 0 | Disponibles: 15
ID: 101 | Clave: P00101 | Nombre: Programación Orientada a Objetos | Docente: Ing. Miguel | Cupo: 25 | Inscritos: 0 | Disponibles: 25
ID: 102 | Clave: P0102 | Nombre: Programación II | Docente: Ing. Carlos | Cupo: 25 | Inscritos: 1 | Disponibles: 24
ID: 105 | Clave: B0105 | Nombre: Bases de Datos | Docente: Mtra. Laura | Cupo: 30 | Inscritos: 2 | Disponibles: 28
ID: 108 | Clave: E0108 | Nombre: Estructura de Datos | Docente: Ing. Ana | Cupo: 30 | Inscritos: 2 | Disponibles: 28
ID: 110 | Clave: V00110 | Nombre: Desarrollo Web | Docente: Mtra. Sofia | Cupo: 20 | Inscritos: 0 | Disponibles: 20
```

Captura 5. Ordenamientos

Seleccione una opción: 10

MATRIZ DE AVANCE (1 = relación)

	105	102	108	101	110	1	2	3	4	35
105	0	0	0	0	0	0	0	0	0	0
102	0	0	0	0	0	0	0	0	0	0
108	0	0	0	0	0	0	0	0	0	0
101	0	0	0	0	0	0	0	0	0	0
110	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0
35	0	0	0	0	1	0	0	0	0	0

Relaciones: fila = curso origen, columna = curso destino.

Captura 6. Búsquedas

```
=====
Seleccione una opción: 15
ID a buscar: 110
Encontrado: ID: 110 | Clave: WEB10 | Nombre: Desarrollo Web | Docente: Hra. Sofia | Cupo: 20 | Inscritos: 0 | Disponibles: 20
Pasos: 5
```

Captura 7. Historial

```
===== HISTORIAL DE ACCIONES =====
1. Sistema iniciado con 5 cursos de ejemplo.
2. Se agregó el curso 1 - Batallas de aura
3. Se agregó el curso 2 - Creeper vs zombie
4. Se inscribió un estudiante en el curso 1
5. Se inscribió un estudiante en el curso 2
6. Se inscribió un estudiante en el curso 2
7. Se agregó el curso 3 - Ramen chino
8. Se inscribió un estudiante en el curso 3
9. Se agregó el curso 4 - Carnicería chapala
10. Se inscribió un estudiante en el curso 4
11. Se agregó el curso 35 - La hora pico
12. Se reconstruyó el árbol binario con los cursos actuales.
13. Se reconstruyó el árbol binario con los cursos actuales.
14. Se creó relación 35 -> 110
15. Se ejecutó Bubble Sort directo.
16. Se ejecutó Bubble Sort inverso.
17. Se ejecutó Inserción directa.
18. Se ejecutó Selección directa.
19. Búsqueda secuencial del ID 110
20. Búsqueda binaria del ID 101
```

16. Conclusión

El desarrollo del Sistema de Gestión de Cursos UTC 3.0 permitió integrar diferentes estructuras de datos y algoritmos dentro de una aplicación funcional desarrollada en Java.

El árbol binario de búsqueda permite organizar y localizar cursos utilizando su identificador. El grafo permite representar relaciones académicas entre diferentes cursos mediante una matriz de adyacencia.

Los métodos de ordenamiento permiten organizar los cursos utilizando diferentes estrategias, mientras que la búsqueda secuencial y la búsqueda binaria permiten comparar distintas formas de localizar información.

La funcionalidad adicional de comparación de pasos permite observar la cantidad de operaciones necesarias para encontrar un curso mediante los diferentes métodos de búsqueda.

Con este proyecto se integran los principales conocimientos correspondientes al tercer parcial de Estructura de Datos, manteniendo una estructura orientada a objetos y una ejecución completamente desde consola.

17. Referencia del proyecto

El contenido y los requisitos utilizados para desarrollar este documento se basan en el archivo:

Proyecto Integrador Parcial 3 - Estructura de Datos - UTC

El proyecto solicita la integración de árboles binarios de búsqueda, grafos, métodos de ordenamiento, métodos de búsqueda, validaciones, historial, casos de prueba, documentación y evidencia de funcionamiento.